

Compiler Design

CSE - 420

Assignment 03

FALL 2025

Note that:

- 1. You will have to submit the hard copy*
- 2. Make an cover page which contains you name, student ID and section*

SDT:

```

P -> {counter = 0; S.next = newlabel("s_"); top = new SymbolTable(); off = 0; offStack = stack(); tabStack = stack();}
      S {P.code = S.code || Label(S.next);}
S -> assign {S.code = assign.code; }
assign -> id = E; {assign.code = E.code || gen(id.lexeme '=' E.addr);}
assign -> L = E; {assign.code = E.code || L.code || gen(L.addr.base '[' L.addr ']' '=' E.addr);}
E -> E1 arith E2 {E.addr=newtemp("e_"); E.code = E1.code || E2.code || gen(E.addr '=' E1.addr arith.op E2.addr);}
E -> id {E.addr = id.lexeme; E.code = "";}
E -> num {E.addr = num.value; E.code = "";}
E -> L {E.addr = newtemp("a_"); E.code = L.code || gen(E.addr '=' L.array.base '[' L.addr '']);}
L -> id [ E ] {L.array = top.get(id.lexeme); L.type = L.array.type.elem; L.addr = newtemp("i_");
               L.code = E.code || gen(L.addr '=' E.addr '*' L.type.width); }
L -> L1 [ E ] {L.array = L1.array; L.type = L1.type.elem; t = newtemp("t_"); L.addr = newtemp("i_");
               L.code = E.code || L1.code || gen(t '=' E.addr '*' L.type.width); } || gen(L.addr '=' L1.addr '+' t);}
S -> D
S -> {S1.next = newlabel("s_"); S2.next = S.next;} S1S2 {S.code = S1.code || label(S1.next) || S2.code; }
S -> if {B.true = newlabel("bt_"); B.false=S1.next=S.next;} (B) {S1} {S.code = B.code || Label(B.true) || S1.code;}
S -> {B.true = newlabel("bt_"); B.false=S.next; S1.next = newlabel("lo_");} while (B) {S1}
      {S.code = label(S1.next) || B.code || label(B.true) || S1.code || gen('goto' S1.next);}
S -> {B.true = newlabel("bt_"); B.false= newlabel("bf_"); S1.next = S2.next = S.next;}
      if (B) {S1} else {S2} {S.code = B.code || label(B.true) || S1.code || gen('goto' S.next) || label(B.false) || S2.code;}
B -> E1 rel E2 {B.code = E1.code || E2.code || gen('if' E1.addr rel.op E2.addr 'goto' B.true) || gen('goto' B.false);}
B -> {B1.true = B.false; B1.false=B.true;} !B1 {B.code =B1.code;}
B -> {B1.true = B.true; B1.false = newlabel("bf_"); B2.true = B.true; B2.false = B.false;}
      B1 || B2 {B.code = B1.code || label(B1.false) || B2.code;}
B -> {B1.true = newlabel("bt_"); B1.false = B.false; B2.true = B.true; B2.false = B.false;}
      B1 && B2 {B.code = B1.code || label(B1.false) || B2.code;}
B -> {B1.true = B.true; B1.false = B.false;} (B1) {B.code = B1.code;}
D -> T id {symbol = new Symbol('name': id.lexeme); symbol.type = T.type; symbol.width = T.width; symbol.offset = off;
           top.insert(id.lexeme, symbol); off = off + T.width; } ; D
D -> ε
T -> B {C.bT = B.type; C.bW=B.width;} C {T.type = C.type; T.width = C.width}
B -> int {B.type = int; B.width = 4;}
B -> float {B.type = float; B.width = 8;}
C -> [num] {C1.bT = C.bT; C1.bW=C.bW;} C1 {C.type = array(num.value, C1.type); C.width = C1.width * num.value;}
C -> ε {C.type = C.bT; C.width = C.bW;}
T -> record {tabStack.push(top); offStack.push(off); top = new SymbolTable(); off = 0;}
           {D} {T.type = top; T.width = off; off = offStack.pop(); top = tabStack.pop();}

```

Note that || means string concatenation in SDT rules

<pre> func newlabel(prefix) { label = prefix counter; counter++; return label; } </pre>	<pre> func newtemp(prefix) { temp = prefix counter; counter++; return temp; } </pre>	<pre> func label(label) { return label ":"; } </pre>	<pre> func gen(params[]) { str = ""; for (param in params) str = str param return str; } </pre>
--	---	---	--

1. Using the SDT, generate the parse tree for the following declaration sequence. Annotate the parse tree nodes with proper values of off variable in different D nodes, proper values of type and width for different B, C, and T nodes (for brevity, you can use t for Type and w for indicating width and write t=sTable for record type variables to avoid cluttering your diagram), and show the content of various symbol tables.

```
int x;  
record {  
    int [40] a;  
    float ab;  
} y;  
int i;
```

2. Using the variable declarations of Question 1, generate the three address codes for the following C code snippet by first drawing the parse tree, annotating each B and S node of the tree with proper values of label attributes, and annotating the E nodes with proper values of address attribute. (Note that the labels and addresses are generated in the same tree traversal), then finally write the three address codes for B, S, and P nodes using those labels and addresses. Be careful, your generated code and attributes must agree with the SDT rules given.

```
while ( x != 0 || i >= x + 10) {  
    x = a[i * 2];  
    i = i + 1;  
} a[x] = i - x;
```